

Detective.be — Agent IA email

Bible technique et opérationnelle du projet

Client	Daniel Hurchon — Detective.be
Marques	Detective Belgique · Detective Belgium · DPDH Investigations
Intégrateur	Cyril Dal — Digital HS (cdal@digitalhs.biz)
Hébergement	VPS Hostinger KVM8 (production) / Mac local (développement)
LLM principal	Kimi K2 via Ollama Pro · LiteLLM · OpenRouter (fallback)
Date d'édition	13 May 2026
Version	1.0 — MVP cadré

Document confidentiel — usage interne projet uniquement.

Table des matières

Table des matières	2
1. Résumé exécutif	4
Chiffres clés	4
2. Contexte business	5
2.1 Le client	5
2.2 Les 3 marques	5
2.3 Typologie des mails entrants	5
2.4 Sensibilités métier (garde-fous éditoriaux)	5
3. Architecture cible	6
3.1 Flux de bout en bout	6
3.2 Sous-systèmes (briques modulaires)	6
4. Stack technique	7
5. Cœur intelligent — RAG + Style Daniel	8
5.1 Bootstrap embeddings (one-shot)	8
5.2 Bootstrap personality (one-shot, reproductible)	8
5.3 Génération à chaque demande client	8
6. Prompts complets (copier-coller)	9
6.1 Prompt classification (app/prompts/classifier_prompt.txt)	9
6.2 Prompt personnalité Daniel (placeholder, à régénérer en S1)	9
6.3 Prompt extraction personnalité (scripts/extract_personality.py)	10
6.4 Prompt génération de brouillon (template assemblé)	10
7. Configuration .env (copiable)	12
8. Layout filesystem	14
8.1 En production (KVM8)	14
8.2 En développement local (Mac Cyril)	14
9. Workflow de livraison MVP	15
10. Supervision 24/7	15
11. Sécurité & garde-fous	16
11.1 Secrets et accès	16
11.2 RGPD et confidentialité	16
12. Coûts mensuels	16
13. Roadmap (S1 → V3)	17

Phase 0 — Brainstorm & cadrage (terminée 2026-05-13)	17
S1 — Infra & data (en cours)	17
S2 — Pipeline ingestion IMAP	17
S3 — Cœur intelligent : RAG + génération	17
S4 — Production sur KVM8 + supervision	18
V2 — Bascule Drafts IMAP + feedback loop	18
V3 — Extensions	18
14. Vérification end-to-end (10 tests avant livraison MVP)	19
15. Setup local & déploiement	20
15.1 Setup local (Mac Cyril)	20
15.2 Déploiement KVM8 (S4)	20
16. Cheatsheet commandes utiles	21
16.1 Développement	21
16.2 Production (sur KVM8)	21
17. Glossaire	22

1. Résumé exécutif

Daniel Hurchon, détective privé belge, gère seul 3 boîtes mail Infomaniak (3 marques) et répond personnellement à environ 50 mails par jour mêlant demandes clients, factures, newsletters et spam. Le projet livre un agent IA Python qui surveille ces boîtes en continu, classifie les mails entrants, et génère un brouillon de réponse « à la Daniel » **uniquement pour les demandes clients**, en s'appuyant sur 1200 paires Q/R historiques anonymisées (RAG sur sqlite-vec).

Au MVP, les brouillons sont envoyés par email à Cyril Dal (intégrateur) via Resend pour validation qualité avant transfert à Daniel. La bascule vers dépôt direct dans le dossier Drafts IMAP de Daniel est planifiée en V2, après deux à quatre semaines de calibration.

Chiffres clés

Boîtes surveillées	3 (Infomaniak)
Volume traité	≈ 50 mails/jour
Fréquence polling	5 minutes
Langues supportées	FR / NL / EN (détection + réponse même langue)
Délai cible MVP	4 semaines (S1 → S4)
Coût mensuel estimé	≈ 25-30 € (Ollama Pro + OpenRouter ponctuel + backup)
Hébergement	VPS Hostinger KVM8 (8 vCPU / 32 Go RAM)

2. Contexte business

2.1 Le client

Daniel Hurchon dirige seul un cabinet d'enquêtes privées. Pas de collaborateur intermédiaire, pas d'assistant. Il lit et répond aux mails depuis Outlook ou Apple Mail sur Mac. L'agent ne le remplace pas, il lui prépare le travail.

2.2 Les 3 marques

Marque	Email	Langue par défaut	Public
Detective Belgique	contact@detectivebelgique.be	FR	francophones BE
Detective Belgium	contact@detectivebelgium.com	EN	international + NL
DPDH Investigations	info@dpdhuinvestigations.be	FR	dossiers spécifiques

2.3 Typologie des mails entrants

- **Demandes clients** (cible prioritaire MVP) : prospects, devis, suivis de dossier, relances
- **Factures / compta** : fournisseurs, OVH, Infomaniak, comptable
- **Newsletters** : marketing B2B, lettres d'info pro
- **Spam / phishing** : faux clients, tentatives d'arnaque
- **Urgences** : situation client critique, deadline imminente
- **Autre** : notifications systèmes, confirmations automatiques

2.4 Sensibilités métier (garde-fous éditoriaux)

- **Confidentialité** : rappels discrets, jamais de divulgation d'autres dossiers
- **Pas d'engagement légal/contractuel par email** : prix, délais → renvoyer vers appel/RDV
- **Neutralité absolue** : pas de jugement sur la situation décrite
- **Vouvoiement par défaut**, tutoiement très rare
- **Concision** : Daniel répond court, va à l'essentiel
- **Toujours une porte de sortie vers un échange humain**

3. Architecture cible

3.1 Flux de bout en bout

```
[3 boîtes Infomaniak IMAP via app passwords]
| polling 5 min (aioimaplib)
v
[Worker Python asyncio, 1 task par boîte]
| nouveau mail non-flaggé $AgentProcessed
v
[Pipeline]
1. Pré-filtre règles -> headers (List-Unsubscribe), expéditeurs
   connus -> tag IMAP + skip
2. Classification LLM -> 6 catégories (Kimi K2 via LiteLLM)
3. Si != demande_client -> tag IMAP catégorie + skip
4. Si demande_client :
   a. Détection langue (fasttext FR/NL/EN)
   b. Embedding du mail (multilingual-e5-large local CPU)
   c. RAG -> top 5 paires Q/R similaires (sqlite-vec)
   d. Génération brouillon (Kimi K2 par défaut)
   |
   v
[Resend API] -> email formaté à cda@digitalhs.biz
[IMAP STORE] flag $AgentProcessed sur le mail entrant
|
v
[Cyril valide -> forward à Daniel pour envoi]
```

3.2 Sous-systèmes (briques modulaires)

#	Sous-système	Rôle
1	Worker IMAP	Polling + détection nouveaux mails + flag idempotence
2	Pré-filtre règles	Court-circuit sur newsletters / billing évidents
3	Classifier LLM	6 catégories à partir du mail brut
4	Détection langue	fasttext FR/NL/EN, défaut = langue de la boîte
5	RAG retrieval	Top-K paires Q/R similaires depuis sqlite-vec
6	Generator	Assemblage prompt système + few-shot + appel LLM
7	Delivery (Resend)	Email HTML formaté → Cyril (MVP)
8	Healthcheck + alertes	FastAPI /health + bot Telegram

4. Stack technique

Couche	Choix	Justification
Runtime	Python 3.11+	Écosystème mail/RAG/LLM riche
Concurrence	asyncio (1 task/boîte)	Suffisant pour 50 mails/jour
IMAP	aiimaplib	Async, gère reconnexion
LLM router	LiteLLM (proxy OpenAI-compatible)	Switch facile Kimi K2 / OpenRouter
LLM principal	Kimi K2 via Ollama Pro (20€/mois)	Qualité top, coût plafonné
LLM fallback	OpenRouter (Claude / GPT-4o)	Spécialisation par tâche
Embeddings	intfloat/multilingual-e5-large	Gratuit, local CPU, FR/NL/EN excellent
Vector store	sqlite-vec	Vit dans les DB existantes
Détection langue	fasttext (lid.176.bin)	Local, instantané
Email outbound	Resend API	Simple, free tier suffisant
État/queue/logs	agent_state.db (SQLite)	Pas besoin de Redis
Config / secrets	.env + pydantic-settings	Single-tenant, chmod 600
Service prod	systemd unit	Natif Linux, pas de Docker au MVP
Healthcheck	FastAPI 127.0.0.1:8765	Sondé par systemd timer
Alertes	Bot Telegram	Gratuit, push immédiat
Backup	Cron → Backblaze B2 (chiffré age)	Restore-tested
Logs	structlog JSON + journalctl	Rotation 7 jours

■ Ne PAS introduire sans discussion explicite : Docker, Celery, Redis, Postgres, Kubernetes, ORM lourd. L'architecture est volontairement légère pour rester maintenable par une seule personne.

5. Cœur intelligent — RAG + Style Daniel

5.1 Bootstrap embeddings (one-shot)

Le script `scripts/bootstrap_embeddings.py` indexe les 1200 paires [mail entrant → réponse de Daniel] dans une table `pairs_vec` (extension `sqlite-vec`) à l'intérieur de chacune des 3 DB SQLite existantes. Les embeddings sont calculés via `multilingual-e5-large` (préfixe « `passage:` » conformément à `e5`).

5.2 Bootstrap personality (one-shot, reproducible)

Le script `scripts/extract_personality.py` échantillonne ~50 réponses représentatives, demande au LLM de produire un guide de style « personnalité Daniel » (ton, formules, longueur, signatures par marque, règles à respecter), et écrit le résultat dans `app/prompts/personality_daniel.txt`. À ré-exécuter quand le corpus s'enrichit.

5.3 Génération à chaque demande client

Pour chaque mail classé `demande_client`, le générateur :

- détecte la langue du mail entrant (FR/NL/EN)
- calcule l'embedding query du mail (préfixe « `query:` »)
- récupère les **top 5** paires Q/R les plus similaires dans la DB de la boîte d'origine
- assemble un prompt système (personnalité Daniel + marque + langue obligatoire)
- ajoute le contexte RAG few-shot puis le mail à traiter
- appelle Kimi K2 via LiteLLM (fallback OpenRouter automatique en cas d'échec)

6. Prompts complets (copier-coller)

Tous les prompts sont en monospace pour faciliter le copier-coller depuis ce PDF. Les versions sources vivent dans `app/prompts/` et dans les scripts.

6.1 Prompt classification (`app/prompts/classifier_prompt.txt`)

Tu es un classifieur d'emails pour un cabinet de détectives privés.

Catégories possibles (renvoie EXACTEMENT un seul mot, en minuscules) :

- `demande_client` : nouveau prospect ou client existant qui pose une question, demande un devis, donne suite à un dossier, sollicite un service.
- `facture` : facture entrante d'un fournisseur, relance de paiement, justificatif comptable.
- `newsletter` : communication marketing, lettre d'info, promotion commerciale.
- `spam` : message non sollicité, phishing, arnaque, contenu manifestement frauduleux.
- `urgent` : situation nécessitant une action immédiate (urgence client, problème critique, deadline imminente explicite).
- `autre` : tout le reste (échanges internes, confirmations automatiques, notifications systèmes).

Règles :

- En cas de doute entre ``demande_client`` et ``urgent``, choisis ``urgent`` uniquement si l'urgence est explicite (mots comme "urgent", "immédiat", "asap", "aujourd'hui", "très important").
- Une demande de devis = ``demande_client``.
- Une notification automatique d'un service tiers (Stripe, LinkedIn, Google, etc.) = ``autre``.

Email à classer :

De : {sender}

Sujet : {subject}

Corps : {body}

Réponds par UN SEUL MOT parmi : `demande_client`, `facture`, `newsletter`, `spam`, `urgent`, `autre`.

6.2 Prompt personnalité Daniel (placeholder, à régénérer en S1)

Ce contenu est un placeholder. Le script `extract_personality.py` le remplacera par un guide de style généré à partir des 1200 paires.

Tu es Daniel Hurchon, détective privé belge dirigeant un cabinet d'enquêtes. Tu réponds personnellement à tes clients en français, néerlandais ou anglais selon la langue du message reçu.

Style général :

- Ton professionnel, direct, courtois mais pas obséquieux
- Pas de jargon excessif, on parle à des clients qui n'ont jamais eu affaire à un détective
- Confidentialité toujours en avant (rappels discrets sur la discrétion)
- Tutoiement rare, vouvoiement par défaut
- Réponses concises, qui invitent à un appel ou un rendez-vous quand le sujet est sensible

Règles fixes :

- TOUJOURS répondre dans la langue du mail entrant.

- TOUJOURS terminer par une signature au nom du cabinet/marque indiqué.
- JAMAIS citer de prix ferme par email pour des dossiers complexes – proposer un appel.
- JAMAIS prendre d'engagement légal/contractuel par email.
- JAMAIS divulguer d'informations sur d'autres dossiers ou clients.

Format attendu : corps de réponse uniquement, prêt à coller dans un email.
Pas de "Sujet:", pas de markdown.

6.3 Prompt extraction personnalité (scripts/extract_personality.py)

Tu es un expert en analyse stylistique. Voici {n} réponses écrites par Daniel Hurchon, détective privé belge, à des clients en français/néerlandais/anglais.

Produis un GUIDE DE STYLE concis et opérationnel (max 600 mots), destiné à être utilisé comme system prompt pour un LLM qui imitera Daniel. Le guide doit couvrir :

1. Ton général (formel/informel, chaleureux/distant, etc.)
2. Formules d'ouverture récurrentes (par langue)
3. Formules de clôture récurrentes (par langue)
4. Vocabulaire caractéristique et tics de langage
5. Longueur typique des réponses (courte/moyenne/longue)
6. Signature(s) utilisée(s) (par marque si différentes)
7. Sujets sensibles : comment Daniel les évite ou les dévie (ex : prix, engagements légaux)
8. Règles à respecter ABSOLUMENT (toujours / jamais)

Format de sortie : texte brut, prêt à être un system prompt. Pas de markdown.

```
--- ÉCHANTILLON DES RÉPONSES ---
{sample}
```

6.4 Prompt génération de brouillon (template assemblé)

Construit dynamiquement par app/pipeline/generator.py à chaque demande client. Voici le template complet :

```
[SYSTEM]
{contenu de personality_daniel.txt}
```

```
Marque/boîte source : {brand}
Langue de réponse OBLIGATOIRE : {language}
```

```
[USER]
Cas #1 (similarité 0.87, langue fr):
--- Mail entrant ---
{paire1.incoming}
--- Réponse de Daniel ---
{paire1.response}
```

```
Cas #2 (similarité 0.81, langue fr):
--- Mail entrant ---
{paire2.incoming}
--- Réponse de Daniel ---
{paire2.response}
```

```
(... top-5 paires)
```

```
--- NOUVEAU MAIL À TRAITER ---
```

De : {sender}
Sujet : {subject}
Corps :
{body}

Génère UN brouillon de réponse en {language}, signé au nom de {brand}, dans le style de Daniel illustré par les cas ci-dessus. Renvoie UNIQUEMENT le corps du message, sans préambule, sans 'Sujet:', sans markdown.

7. Configuration .env (copiable)

Copier .env.example vers .env et remplir les valeurs. Ne JAMAIS commiter le .env (le .gitignore le bloque).

```
# --- Infomaniak IMAP (3 boîtes) ---
IMAP_HOST=mail.infomaniak.com
IMAP_PORT=993

MAILBOX_1_NAME=detective_belgique
MAILBOX_1_USER=contact@detectivebelgique.be
MAILBOX_1_APP_PASSWORD=
MAILBOX_1_BRAND=Detective Belgique
MAILBOX_1_DEFAULT_LANG=fr

MAILBOX_2_NAME=detective_belgium
MAILBOX_2_USER=contact@detectivebelgium.com
MAILBOX_2_APP_PASSWORD=
MAILBOX_2_BRAND=Detective Belgium
MAILBOX_2_DEFAULT_LANG=en

MAILBOX_3_NAME=dpdh_investigations
MAILBOX_3_USER=info@dpdhuinvestigations.be
MAILBOX_3_APP_PASSWORD=
MAILBOX_3_BRAND=DPDH Investigations
MAILBOX_3_DEFAULT_LANG=fr

# --- LLM (LiteLLM router) ---
OLLAMA_PRO_API_KEY=
OLLAMA_PRO_BASE_URL=https://ollama.com/api
LLM_MODEL_DEFAULT=ollama_chat/kimi-k2
OPENROUTER_API_KEY=
LLM_MODEL_FALLBACK=openrouter/anthropic/claude-sonnet-4
LLM_MODEL_CLASSIFIER=ollama_chat/kimi-k2

# --- Resend ---
RESEND_API_KEY=
RESEND_FROM=agent@digitalhs.biz
DRAFT_RECIPIENT=cdal@digitalhs.biz

# --- RAG ---
EMBEDDING_MODEL=intfloat/multilingual-e5-large
RAG_TOP_K=5

# --- Polling ---
POLL_INTERVAL_SECONDS=300

# --- Stockage ---
DATA_DIR=./data
DB_BOITE_1=./data/boite1.sqlite
DB_BOITE_2=./data/boite2.sqlite
DB_BOITE_3=./data/boite3.sqlite
DB_AGENT_STATE=./data/agent_state.db

# --- Healthcheck ---
HEALTHCHECK_HOST=127.0.0.1
HEALTHCHECK_PORT=8765

# --- Telegram alertes (optionnel V1) ---
TELEGRAM_BOT_TOKEN=
TELEGRAM_CHAT_ID=

# --- Logs ---
```

LOG_LEVEL=INFO

8. Layout filesystem

8.1 En production (KVM8)

```
/opt/detective-agent/
■■■ app/
■   ■■■ main.py                # entrypoint asyncio
■   ■■■ config.py              # pydantic-settings
■   ■■■ healthcheck.py         # FastAPI /health
■   ■■■ workers/imap_poller.py # 1 task asyncio par boîte
■   ■■■ pipeline/
■   ■   ■■■ prefilter.py        # règles headers/expéditeurs
■   ■   ■■■ classifieur.py      # LLM 6 catégories
■   ■   ■■■ language.py         # fasttext FR/NL/EN
■   ■   ■■■ rag.py              # embed + retrieve sqlite-vec
■   ■   ■■■ generator.py        # assemblage prompt + LLM call
■   ■■■ delivery/resend_notifier.py
■   ■■■ llm/router.py           # wrapper LiteLLM + fallback
■   ■■■ prompts/
■       ■■■ classifieur_prompt.txt
■       ■■■ personality_daniel.txt
■■■ data/
■   ■■■ boîte1.sqlite           # DB existantes anonymisées
■   ■■■ boîte2.sqlite
■   ■■■ boîte3.sqlite
■   ■■■ agent_state.db          # queue, logs, télémétrie
■■■ scripts/
■   ■■■ bootstrap_embeddings.py
■   ■■■ extract_personality.py
■   ■■■ build_bible_pdf.py
■■■ deploy/detective-agent.service
■■■ .env                        # chmod 600
■■■ venv/
■■■ logs/
```

8.2 En développement local (Mac Cyril)

Identique, à la racine /Users/cda1/DEV_APP_CLAUDE/DETECTIVE_BE/.

9. Workflow de livraison MVP

Le brouillon est envoyé par email à `cdal@digitalhs.biz` via Resend, formaté comme suit :

- **Sujet** : `[AGENT][${marque}] ${sujet du mail original}`
- **Corps HTML** :
 - Métadonnées : boîte source, expéditeur, date reçue, langue détectée, modèle utilisé
 - Brouillon proposé (encadré, copiable, monospace)
 - Mail original intégral (encadré jaune)
 - Top 3 cas RAG utilisés (extraits + scores similarité)

Cyril relit, ajuste si besoin, transmet à Daniel via forward standard. Quand la qualité est stabilisée (mesurée sur 2-4 semaines), bascule en V2 vers dépôt IMAP direct dans le dossier Drafts natif de Daniel.

10. Supervision 24/7

- **systemd unit** avec `Restart=always, RestartSec=10`
- **Healthcheck** : endpoint `/health` (FastAPI sur `127.0.0.1:8765`) renvoie OK si toutes les connexions IMAP actives + dernier cycle `< 10 min`
- **systemd timer** sonde `/health` chaque minute, déclenche restart si KO
- **Bot Telegram** envoie alertes pour : agent down, IMAP timeout `> 3` tentatives consécutives, échec génération `> 5/heure`, taux d'erreur LLM `> 10%`
- **Logs structurés JSON** dans `journalctl`, rotation 7 jours
- **Tableau de bord léger** (V1.5) : page HTML statique générée par cron, accessible via SSH tunnel

11. Sécurité & garde-fous

11.1 Secrets et accès

- 3 app passwords Infomaniak + clé Resend + clés Ollama/OpenRouter dans .env (chmod 600, propriétaire detective-agent en prod)
- TLS partout : IMAPS 993, SMTP 587 STARTTLS, HTTPS pour API LLM
- Healthcheck bind 127.0.0.1 uniquement, aucune API exposée publiquement
- Backups chiffrés (age) avant push vers Backblaze
- App passwords séparés permettent révocation individuelle par boîte

11.2 RGPD et confidentialité

■ Pas de log du contenu intégral des mails en clair — uniquement IDs et métadonnées. Pour debug, ajouter un flag explicite LOG_MAIL_BODY=true.

■ Les 3 DB SQLite anonymisées restent sensibles : ne jamais commit dans git, ne jamais uploader en clair vers un service tiers. Le .gitignore les bloque déjà.

■ Ne jamais écrire dans les vraies boîtes Infomaniak en dev. Mode --dry-run obligatoire tant que la calibration n'est pas validée.

12. Coûts mensuels

Poste	Coût mensuel
VPS Hostinger KVM8	déjà payé
Infomaniak (3 boîtes)	déjà payé
Ollama Pro (Kimi K2)	20 €
OpenRouter (fallback ponctuel)	< 5 €
Resend (free tier 3000 mails/mois)	0 €
Backblaze B2 (~5 Go)	< 1 €
Bot Telegram	0 €
Total estimé	≈ 25-30 €

13. Roadmap (S1 → V3)

État courant : Phase 0 (brainstorm + scaffolding) terminée. S1 démarre dès que les pré-requis bloquants côté Cyril sont fournis (DB SQLite, schéma, .env rempli).

Phase 0 — Brainstorm & cadrage (terminée 2026-05-13)

- ✓ Spec technique figée (docs/SPEC.md)
- ✓ Choix LLM : Kimi K2 via Ollama Pro + LiteLLM
- ✓ Choix vector store : sqlite-vec
- ✓ Choix livraison MVP : Resend → cdal@digitalhs.biz
- ✓ Scaffolding code complet en place

S1 — Infra & data (en cours)

Pré-requis bloquants :

- Déposer les 3 fichiers .sqlite anonymisés dans data/
- Partager le schéma de chaque DB (sqlite3 ... ".schema")
- Remplir .env avec : 3 app passwords Infomaniak, OLLAMA_PRO_API_KEY, RESEND_API_KEY
- Vérifier le domaine Resend agent@digitalhs.biz

Tâches code :

- Créer venv + installer deps (pip install -e .[dev])
- Smoke test LLM : appel Kimi K2 via LiteLLM
- Smoke test embeddings : charger e5-large, encoder, vérifier dim
- Smoke test sqlite-vec : indexer 5 vecteurs jouets, retrieve top-1
- Adapter scripts/bootstrap_embeddings.py au schéma réel
- Adapter scripts/extract_personality.py au schéma réel
- Exécuter bootstrap_embeddings → vérifier pairs_vec peuplé
- Exécuter extract_personality → relire et valider personality_daniel.txt

S2 — Pipeline ingestion IMAP

- Implémenter imap_poller._poll_once (aioimaplib)
- Mode --dry-run sans flag
- Reconnexion automatique IMAP
- Tests sur 1 boîte d'abord, puis les 3
- Persister classifications dans agent_state.db
- Tests unitaires : mock IMAP + 5 mails-fixtures

S3 — Cœur intelligent : RAG + génération

- Brancher language.detect_language sur les mails entrants

- Brancher rag.retrieve sur la vraie DB
- Brancher generator.generate_draft end-to-end
- Brancher delivery.resend_notifier.notify_draft
- Calibration qualité sur 50 mails réels
- Vérification multilingue FR/NL/EN
- Vérification signatures par marque

S4 — Production sur KVM8 + supervision

- Setup KVM8 : user dédié, Python 3.11, venv, /opt/detective-agent/
- Installer .env prod (chmod 600)
- Installer systemd unit, systemctl enable --now
- Bot Telegram + chat_id
- systemd timer healthcheck
- Cron backup quotidien Backblaze (chiffré age)
- Procédure restore documentée et testée
- Documentation opérationnelle (restart, restore, ajout boîte)
- Lancement officiel + monitoring 1 semaine

V2 — Bascule Drafts IMAP + feedback loop

- Module delivery/imap_drafts.py (IMAP APPEND dans Drafts)
- Switch config DELIVERY_MODE=resend|imap_drafts
- Capture feedback (diff brouillon vs envoyé)
- Tableau de bord taux d'acceptation + distance d'édition

V3 — Extensions

- Module factures (extraction structurée + tâche compta)
- Bot WhatsApp client (Twilio / WhatsApp Business API)
- Dashboard web supervision
- Suppression mails > 28 jours (rétention RGPD)
- Architecture multi-sub-agents avec LLM différencié

14. Vérification end-to-end (10 tests avant livraison MVP)

#	Test	Critère de succès
1	IMAP	Injecter mail test depuis compte externe vers chacune des 3 boîtes → détection < 5 min + tag \$AgentPro
2	Classification	1 mail de chaque catégorie (newsletter, facture, demande client, spam) → classification correcte dans log
3	RAG	Mail demande client similaire à un cas historique → top-5 contient le cas attendu
4	Génération	Brouillon respecte langue du mail entrant + signature de la bonne marque
5	Livraison	Email reçu sur cda1@digitalhs.biz avec format complet (métadonnées + brouillon + original + cas RAG)
6	Multilingue	3 mails identiques FR/NL/EN → 3 brouillons cohérents chacun dans sa langue
7	Robustesse	Kill brutal du process → systemd redémarre dans 10s, reprend sans doublon (idempotence flag IMAP)
8	Supervision	Couper IMAP volontairement (firewall) → alerte Telegram reçue
9	Backup	Exécuter restore sur copie → DB lisibles
10	Charge	Injecter 100 mails simultanés → tous traités sans crash, latence moyenne < 30s/mail

15. Setup local & déploiement

15.1 Setup local (Mac Cyril)

```
cd /Users/cdal/DEV_APP_CLAUDE/DETECTIVE_BE
python3 -m venv venv
source venv/bin/activate
pip install -e ".[dev]"
cp .env.example .env
# éditer .env avec les vraies clés

# Bootstrap one-shot (S1, après que les DB sont en place)
python -m scripts.bootstrap_embeddings
python -m scripts.extract_personality

# Lancer l'agent
python -m app.main
```

15.2 Déploiement KVM8 (S4)

```
# Sur le VPS, en root
useradd -r -s /usr/sbin/nologin -d /opt/detective-agent detective-agent
mkdir -p /opt/detective-agent
chown -R detective-agent: /opt/detective-agent

# Cloner / rsync le code
rsync -av --exclude=venv --exclude=.env \
  /local/DETECTIVE_BE/ root@vps:/opt/detective-agent/

# Sur le VPS, créer venv et installer
sudo -u detective-agent python3.11 -m venv /opt/detective-agent/venv
sudo -u detective-agent /opt/detective-agent/venv/bin/pip install -e \
  /opt/detective-agent

# Copier .env (chmod 600)
scp .env.prod root@vps:/opt/detective-agent/.env
chmod 600 /opt/detective-agent/.env
chown detective-agent: /opt/detective-agent/.env

# systemd
cp /opt/detective-agent/deploy/detective-agent.service \
  /etc/systemd/system/
systemctl daemon-reload
systemctl enable --now detective-agent
systemctl status detective-agent
journalctl -u detective-agent -f
```

16. Cheatsheet commandes utiles

16.1 Développement

```
# Lancer l'agent
python -m app.main

# Tests
pytest
pytest -k test_classifieur # un sous-ensemble

# Lint / format
ruff check .
ruff format .

# Inspecter le schéma d'une DB
sqlite3 data/boite1.sqlite ".schema"
sqlite3 data/boite1.sqlite ".tables"

# Inspecter sqlite-vec
sqlite3 data/boite1.sqlite "SELECT count(*) FROM pairs_vec;"

# Régénérer la bible PDF
python scripts/build_bible_pdf.py
```

16.2 Production (sur KVM8)

```
# Status
systemctl status detective-agent

# Logs en temps réel
journalctl -u detective-agent -f

# Logs des dernières 24h
journalctl -u detective-agent --since '24 hours ago'

# Restart
systemctl restart detective-agent

# Healthcheck manuel
curl -s http://127.0.0.1:8765/health | jq

# Backup manuel
/opt/detective-agent/scripts/backup.sh

# Tester un restore
/opt/detective-agent/scripts/restore.sh /tmp/restore-test
```

17. Glossaire

Terme	Définition
App password	Mot de passe applicatif Infomaniak distinct du mot de passe principal, permet une révocation par boîte s
\$AgentProcessed	Flag IMAP custom posé sur les mails entrants après traitement par l'agent, pour garantir l'idempotence (
e5-large	Modèle d'embeddings multilingue intfloat/multilingual-e5-large, gratuit, tourne en CPU. Convention : pré
Few-shot RAG	Technique de génération qui injecte 3-5 exemples concrets (paires Q/R retrouvées par similarité) directe
Kimi K2	Modèle de langue de Moonshot AI, accessible via Ollama Pro (abonnement 20€/mois). Performances co
LiteLLM	Bibliothèque Python qui expose une API unique compatible OpenAI mais route vers n'importe quel provi
OpenRouter	Marketplace de LLMs avec une seule API, utilisé ici en fallback et pour tester d'autres modèles à la dem
Resend	Service d'envoi d'emails transactionnels par API HTTP, free tier 3000 mails/mois.
sqlite-vec	Extension SQLite récente qui ajoute des opérateurs vectoriels (recherche par similarité) à n'importe que
structlog	Bibliothèque de logs structurés JSON (vs texte plat), parsable par les outils de monitoring.
systemd unit	Fichier de configuration qui décrit un service à long terme géré par systemd : lancement, supervision, re
Hostinger KVM8	Plan de VPS Hostinger : 8 vCPU, 32 Go RAM, 400 Go NVMe.

Fin de la bible. Pour toute question, ouvrir une session Claude Code dans le dossier DETECTIVE_BE — il chargera CLAUDE.md et saura quoi faire.